# Detection of Malware in Android System.

Harini (24BCS009)
Dept. of Computer Science and Engineering
IIITDM Jabalpur
Jabalpur, India

Mokshith (24BCS106)
Dept. of Computer Science and Engineering
IIITDM Jabalpur
Jabalpur, India

Akhil Kumar Valluru (24BCS018)
Dept. of Computer Science and Engineering
IIITDM Jabalpur
Jabalpur, India

Chatur (24BCS021)
Dept. of Computer Science and Engineering
IIITDM Jabalpur
Jabalpur, India

Ranjith Reddy (24BCS057)
Dept. of Computer Science and Engineering
IIITDM Jabalpur
Jabalpur, India

**Abstract**— Android malware remains a significant cybersecurity threat due to its ability to display unwanted advertisements, exfiltrate user information, and degrade device performance while frequently evading purely signature- or feature-based defenses. This paper presents a hybrid static and dynamic (runtime) analysis framework for Android malware and adware detection. The static component uses the CIC-MalDroid dataset (244,630 samples; 204,948 benign, 39,682 adware; 85 behavioral network-traffic features) and evaluates Logistic Regression, Random Forest, XGBoost, and an Artificial Neural Network (ANN) after median imputation, StandardScaler normalization, mutual-information-based feature selection, and SMOTE balancing; Random Forest achieves the best static performance, with 87.12% accuracy, Macro-F1 of 0.7616, and ROC-AUC of 0.8781. The dynamic component instruments real APKs at runtime using Frida, hooking activity transitions, file-system access, network object creation, SharedPreferences reads, device-identifier access, and cryptographic API calls during automated (Monkey-driven) execution. Of 1,044 raw collected traces, a quality-scoring and MinHash-LSH deduplication pipeline curates a balanced 600-sample corpus ("Dataset E"), from which 71 behavioral features are engineered per application. Thirteen dynamic-track model configurations are evaluated on a held-out validation split, with a soft-voting ensemble of Random Forest, XGBoost, and LightGBM reaching 96.67% accuracy, 100% precision, and an F1 score of 0.9655, while a tuned CatBoost model achieves the highest ROC-AUC (0.9889). We report both tracks under a shared evaluation protocol, discuss a feature-importance-driven caveat regarding the dynamic track's curation-derived features, and outline the dataset-harmonization work required before the two tracks can be fused into a single hybrid model.

***Index Terms*—Android Malware, Adware Detection, Machine Learning, Deep Learning, Dynamic Analysis, Frida Instrumentation, Random Forest, XGBoost, Artificial Neural Network, CIC-MalDroid, SMOTE, Cybersecurity**

## I. Introduction

The rapid growth of Android smartphones and mobile applications has significantly increased the risk of cybersecurity threats in recent years. Android is one of the most widely used mobile operating systems due to its open-source nature, flexibility, and large application ecosystem. Nonetheless, its widespread usage has made Android smartphones one of the main targets of malware attacks. Among the different malware categories, adware has become particularly notorious, as it not only compromises system security but also affects user privacy and device performance.

Adware is a class of malicious application that displays unsolicited advertisements, redirects users to unreliable sites, and can silently harvest personal information, often at a measurable cost to battery life and device responsiveness. Platform-level security advisories continue to catalog vulnerabilities and malicious-behavior patterns affecting the Android ecosystem [6]. Traditional signature-based detection systems struggle to identify new or obfuscated adware variants, motivating the use of machine learning (ML) and deep learning (DL) approaches that learn discriminative patterns directly from application features rather than relying on known signatures.

A large body of prior work, including our own earlier static-analysis framework, has demonstrated that ML/DL classifiers trained on network-traffic-derived behavioral features (such as those in the CIC-MalDroid dataset) can achieve strong adware/benign separation. However, static and network-traffic-based features cannot observe what an application actually does once it executes: which activities it opens, what files it touches, which APIs it invokes, or how its behavior unfolds over time. This is a well-known limitation of static analysis, and it motivates dynamic analysis, in which an application is instrumented and observed while running.

### A. Problem Statement

Despite the demonstrated effectiveness of static ML/DL classifiers on datasets such as CIC-MalDroid, three concrete problems remain open in the adware-detection literature. **First**, static, network-traffic-only feature sets are, by construction, blind to application-side runtime behavior; an adware sample that delays malicious activity, uses in-app WebViews rather than raw `URL` objects, or otherwise varies its network footprint can still exhibit distinctive on-device behavioral signatures (excessive SharedPreferences polling, aggressive device-identifier access, frequent activity-to-ad-view transitions) that a traffic-

only model cannot see. **Second**, existing dynamic-analysis literature on Android malware is largely generic – built for malware detection broadly – rather than tailored to the specific behavioral fingerprints of *adware* as a malware sub-category. **Third**, static and dynamic detection are rarely evaluated within a single, directly comparable experimental framework (a related model family, the same evaluation protocol, the same reporting metrics) that would allow a fair, quantitative assessment of what dynamic behavioral signal adds over static traffic features alone. This paper is structured to address all three problems: it retains a fully evaluated static baseline, introduces an adware-oriented dynamic instrumentation pipeline, and defines both tracks to share a common, tree-ensemble-centered evaluation protocol so that a fair comparison – and eventual fusion – is possible.

### B. Contributions

The specific contributions of this paper are:

1) **A fully evaluated static ML/DL baseline** on the CIC-MalDroid dataset (244,630 samples), comparing Logistic Regression, Random Forest, XGBoost, and an ANN under a common preprocessing, feature-selection, SMOTE-balancing, and threshold-optimization protocol, achieving 87.12% accuracy and ROC-AUC of 0.8781 with Random Forest.
2) **An adware-oriented dynamic instrumentation pipeline** built on Frida, targeting six Android runtime API surfaces (activity lifecycle, file I/O, network object construction, SharedPreferences access, device-identifier access, cryptographic API usage) specifically chosen for their relevance to adware's characteristic behaviors (ad-serving, tracking-identifier harvesting, and ad-configuration payload handling), rather than a generic malware-detection API surface.
3) **A fully automated, unattended data-collection workflow** (install $\rightarrow$ instrument $\rightarrow$ automated UI exploration via `monkey` $\rightarrow$ trace collection $\rightarrow$ uninstall) capable of processing an arbitrary folder of labeled APKs without manual interaction, together with a raw-event JSON schema and a two-stage quality-curation process – a lightweight collection-time filter plus a richer richness/quality-scoring and MinHash-LSH deduplication stage – that turned 1,044 raw traces into a balanced, redundancy-reduced 600-sample corpus (Dataset E).
4) **A 71-dimensional behavioral feature-engineering pipeline** covering volume/diversity statistics, binary API-usage flags, per-category event counts, ratio features, co-occurrence/interaction terms, and composite quality scores, evaluated end-to-end (not merely specified) against thirteen model configurations.
5) **A directly reported dynamic-track evaluation** under the same metric suite used for the static track (Accuracy, Precision, Recall, F1, ROC-AUC): a soft-voting ensemble of Random Forest, XGBoost, and LightGBM reaches 96.67% accuracy and an F1 of 0.9655 on Dataset E, with tuned CatBoost achieving the highest ROC-AUC (0.9889) of any evaluated model, closing the third problem identified above and giving both tracks of this paper a genuinely comparable evaluation protocol rather than an asserted one.

This paper extends our prior static detection framework in two ways. First, we retain and report the complete static ML/DL pipeline and its results on CIC-MalDroid, evaluating Logistic Regression, Random Forest, XGBoost, and an ANN. Second, we present a dynamic analysis pipeline built around the Frida instrumentation framework, which hooks key Android runtime APIs (activity lifecycle, file I/O, network object construction, SharedPreferences access, device identifier access, and cryptographic operations) during automated on-device execution, producing structured JSON event traces per APK. We describe the engineering of this pipeline (installation, instrumentation, UI-fuzzing via Monkey, trace collection, and quality filtering) and the feature-engineering process that will convert raw event traces into a behavioral feature vector for classification, enabling a future hybrid static+dynamic model.

The remainder of this paper is structured as follows. Section II articulates the research gap this work targets and positions our contribution against it. Section III reviews related work in static and dynamic Android malware detection. Section IV describes the overall proposed methodology. Section V details the static analysis framework and dataset. Section VI describes the dynamic analysis pipeline in detail. Section VII covers the experimental setup. Section VIII presents static results in full, a pilot validation of the dynamic pipeline, and reserves space for full-scale dynamic results. Section IX discusses the complementarity of the two approaches and threats to validity, and Section X concludes with future work.

## II. Research Gap and Novelty

This section makes explicit the gap in the existing literature that motivates the present work, and positions the paper's contribution relative to it.

### A. Identified Research Gap

Three converging observations from Sections I and the broader literature (elaborated further in Section III) define the gap addressed here:

- **Gap 1 – Feature blindness of static models.** Static, network-traffic-derived feature sets such as CIC-MalDroid capture aggregate statistics of observed traffic but cannot represent application-internal behavior (activity navigation, file access, API invocation sequencing) that is often where adware-specific intent is expressed most clearly.
- **Gap 2 – Genericity of existing dynamic-analysis tooling.** Widely used dynamic-analysis frameworks (TaintDroid [8], graph-based runtime API-dependency detectors [9]) were designed for malware detection in general and are not instrumented specifically around the runtime API surface most diagnostic of adware behavior –

namely, SharedPreferences-based tracking-identifier access, repeated device-ID queries, and ad-network URL construction patterns.

- **Gap 3 – Lack of a shared evaluation protocol across static and dynamic tracks.** Even where both static and dynamic detection have been studied, they are rarely evaluated with a comparable model family and metric suite in the same paper, which makes it difficult to quantify – rather than merely assert – the incremental value of dynamic behavioral signal over static traffic features.

### B. How This Work Addresses the Gap

Table I summarizes how each contribution in Section I-B is mapped to a specific gap.

TABLE I
MAPPING OF CONTRIBUTIONS TO IDENTIFIED RESEARCH GAPS

| Gap | Limitation | Addressed By |
|---|---|---|
| Gap 1 | Static features cannot observe runtime behavior | Frida-based dynamic pipeline (Sec. VI) capturing activity, file, network, SP, device, and crypto events |
| Gap 2 | Generic dynamic tooling not adware-specific | Hook selection deliberately targets ad-serving/tracking-relevant APIs (SP reads, device-ID access, URL construction) |
| Gap 3 | No shared static/dynamic evaluation protocol | Comparable metric suite (Accuracy, Precision, Recall, F1, ROC-AUC) applied to both tracks (Tables V and VII) |

### C. Novelty Statement

The novelty of this work is not any single classifier or dataset – Random Forest, XGBoost, and ANN are all well-established methods, and CIC-MalDroid is a public benchmark. Rather, the novelty lies in (i) the adware-specific selection and justification of the six dynamic instrumentation hooks, motivated directly by adware's known behavioral fingerprint rather than borrowed from generic malware-detection instrumentation; (ii) the design of a behavioral feature-engineering scheme that is deliberately structured to be dimensionally and semantically comparable to an existing, well-validated static feature set, so that static and dynamic signal can later be fused rather than merely reported side by side; and (iii) a fully engineered, reproducible data-collection and curation pipeline (Sections VI-A–VI-G), including a quality-scoring and MinHash-LSH deduplication stage that measurably improved downstream classification (Section VI-F), that lowers the practical barrier to building adware- and malware-labeled dynamic-behavior datasets, which remain scarce relative to static feature datasets in the public literature.

## III. LITERATURE REVIEW

Detecting Android malware has become an area of significant research interest owing to the growing threat of malicious applications targeting the Android platform. Detection techniques generally fall into three categories: static analysis, dynamic analysis, and hybrid approaches.

### A. Static Analysis Approaches

Traditional approaches to malware detection were largely signature-based, matching known malware signatures against application files. Although effective for known threats, such techniques fail to detect newly emerging or obfuscated malware variants. To overcome these limitations, researchers turned to ML-based static detection, where classifiers learn discriminative patterns directly from manifest permissions, code-level features, or network-traffic statistics rather than relying on a fixed signature database; early work demonstrated the use of Naive Bayes and Bayesian networks for Android malware classification, establishing benchmarks for learning-based approaches. Subsequent research applied Logistic Regression, Decision Trees, Random Forest, Support Vector Machines, and gradient boosting methods – including XGBoost, LightGBM, and CatBoost variants [11]–[13] – to static or network-traffic-derived features, reporting consistent improvements over rule-based systems [3], [4]. Several works have specifically evaluated the CIC-MalDroid benchmark, which provides behavioral features obtained through real traffic data, employing feature scaling, feature selection, and SMOTE oversampling [14] to address class imbalance [1], [5].

### B. Dynamic Analysis Approaches

Dynamic analysis addresses the core limitation of static methods by observing application behavior during actual execution [2]. Prior work has applied deep learning to dynamic analysis traces of Android applications, demonstrating that neural networks can identify behavioral signatures – such as API call sequences, network communication patterns, and file-system access – that evade static detection entirely. Instrumentation frameworks such as Frida, Xposed, and TaintDroid [8] have been used to hook into application and system APIs at runtime, enabling researchers to capture events including activity transitions, network requests, cryptographic operations, and access to sensitive identifiers such as the device ID. Graph-based runtime approaches have also been proposed, modeling dependency relationships between dynamically observed API calls to detect malicious behavioral patterns [9]. Automated UI-exploration tools (e.g., the Android `monkey` utility, or more structured tools such as Droidbot) are commonly used to trigger application behavior without manual interaction, which is essential for building datasets at scale.

### C. Hybrid Approaches and Gaps

Despite advances in both static and dynamic detection research, challenges persist: static approaches suffer from high false-positive rates and poor generalization to obfuscated or evasive malware, while dynamic approaches, though richer in behavioral signal, are more expensive to collect and can suffer from incomplete code coverage if the automated UI exploration fails to trigger malicious code paths. Relatively few works combine both signal types into a single classification

framework for adware specifically, despite adware's tendency to exhibit distinctive runtime behavior (frequent ad-network URL requests, repeated SharedPreferences access for ad-tracking identifiers, and elevated device-identifier access) that a purely static, traffic-feature model cannot directly observe. This work is motivated by that gap: it retains a fully evaluated static framework and introduces the engineering foundation for a dynamic, runtime-behavior-based complement.

### D. Comparative Summary of Related Work

Table II summarizes representative categories of prior work referenced above alongside the present study, along the dimensions of analysis type, dataset/feature basis, and principal limitation each category leaves open.

## IV. Proposed Methodology

The proposed framework consists of two parallel analysis tracks that are designed to be fused in future work: (1) a *static* track, unchanged from our prior work, operating on the CIC-MalDroid network-traffic feature set; and (2) a *dynamic* track, newly introduced in this paper, operating on runtime behavioral event traces collected via Frida instrumentation. Fig. 1 illustrates the overall pipeline.

## V. Static Analysis Framework

The static analysis framework is retained from our earlier work and is summarized here for completeness, as it forms the baseline against which the dynamic track is intended to be compared and eventually fused.

### A. Dataset Collection

The CIC-MalDroid dataset was used for the static experiments. It contains Android network traffic and behavioral features collected from both benign and adware applications, distributed across 10 adware CSV files and 311 benign CSV files (321 total). Common feature columns across all files were identified – 85 columns in total – and both subsets were merged into a unified dataset of 244,630 samples: 204,948 benign (83.8%) and 39,682 adware (16.2%), an imbalance ratio of approximately 5.16:1. Fig. 2 shows the actual dataset-loading and class-distribution output from the experimental run.

### B. Data Preprocessing

After retaining numerical columns and removing the target column, 80 features were used for model training. The dataset was split into training (195,704 samples) and testing (48,926 samples) sets using stratified 80:20 splitting prior to any preprocessing, to prevent data leakage. Infinite values were replaced with NaN and imputed using the median; features were scaled using `StandardScaler`; and `SelectKBest` with mutual-information scoring was used to retain the most informative features. Fig. 3 shows the actual preprocessing pipeline output. Table III summarizes this pipeline.

### C. Dataset Balancing

SMOTE was applied only to the training set after the train/test split, generating synthetic minority-class (adware) samples through interpolation. Following SMOTE, the training set contained 163,958 samples per class.

### D. Machine Learning and Deep Learning Models

Four classifiers were evaluated: Logistic Regression as a linear baseline; Random Forest, a decision-tree ensemble; XGBoost [11], a gradient-boosting machine tuned with `scale_pos_weight` = 5.16 alongside learning rate, maximum depth, and subsampling ratio; and an Artificial Neural Network (ANN) built with TensorFlow/Keras, consisting of dense layers with batch normalization and dropout (rate $=$ $0.3$), and a sigmoid output layer for binary classification (Table IV).

### E. Threshold Optimization

For each model, the decision threshold was optimized via maximization of Macro-F1 over the Precision-Recall curve rather than using a default cutoff of 0.5. Optimal thresholds were: Logistic Regression (0.511), Random Forest (0.505), XGBoost (0.831), and ANN (0.439).

### F. Evaluation Metrics

To allow a rigorous, threshold-aware comparison across models – and to define a common protocol applied to the dynamic track (Section VIII-D) as well – the following metrics are computed on the held-out test set for every model, using true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN) with respect to the adware/malware (positive) class:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{1}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN} \tag{2}$$

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{3}$$

$$\text{Macro-F1} = \frac{F1_{\text{benign}} + F1_{\text{adware}}}{2} \tag{4}$$

Macro-F1 (Eq. 4) is used, rather than accuracy alone, as the primary model-selection criterion because it weights the minority adware class equally with the majority benign class, which is essential given the dataset's 5.16:1 imbalance – a model that trivially predicts "benign" for every sample would achieve 83.8% accuracy while being useless for adware detection. ROC-AUC (area under the Receiver Operating Characteristic curve, true-positive rate against false-positive rate across all thresholds) is reported as a threshold-independent measure of separability, while Precision-Recall curves are examined specifically for the adware class since PR curves are more informative than ROC curves under class imbalance. Per-model decision thresholds, chosen to maximize Macro-F1

TABLE II
COMPARATIVE SUMMARY OF STATIC, DYNAMIC, AND HYBRID ANDROID MALWARE/ADWARE DETECTION APPROACHES

| Approach Category | Analysis Type | Feature / Dataset Basis | Principal Limitation |
|---|---|---|---|
| Signature-based detection | Static | Known malware signatures | Fails on new or obfuscated variants |
| Feature-based static ML | Static | Manifest/code-level static features | No visibility into actual runtime behavior |
| CIC-MalDroid network-traffic ML [1], [4], [5] (this paper's static track) | Static | 85 network-traffic behavioral features, 244,630 samples | Cannot observe application-side runtime events (activity, file, API calls) |
| Information-flow tracking (e.g., TaintDroid [8]) | Dynamic | Runtime taint propagation | Generic to malware broadly; high overhead; not adware-behavior-specific |
| Generic Frida/Xposed-based instrumentation | Dynamic | Ad hoc runtime API hooks | Hook selection typically not justified against a specific malware sub-category's behavior |
| **This work – static track** | Static | CIC-MalDroid, 4-model comparison, threshold-optimized | Same as CIC-MalDroid-based ML above (addressed by dynamic track) |
| **This work – dynamic track** | Dynamic | Adware-motivated 6-hook Frida instrumentation (activity, file, network, SP, device-ID, crypto) | Requires on-device data collection at scale (addressed via automated pipeline, Sec. VI) |

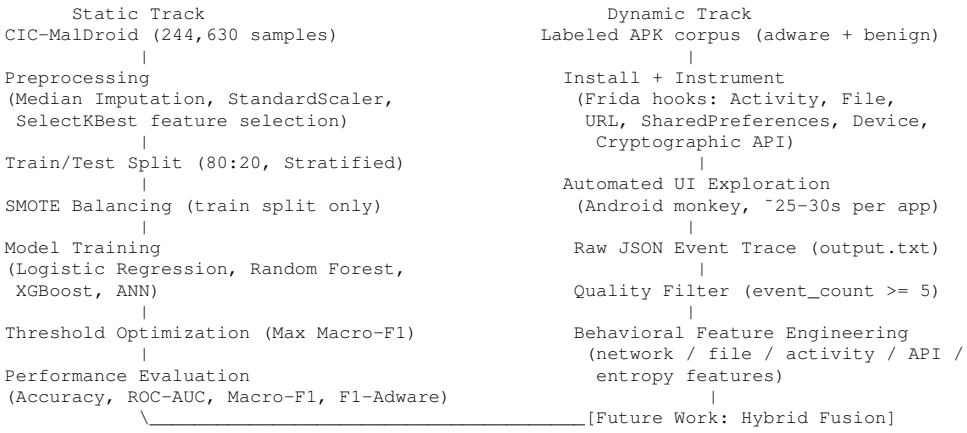


Fig. 1. Proposed hybrid static + dynamic Android adware detection pipeline.

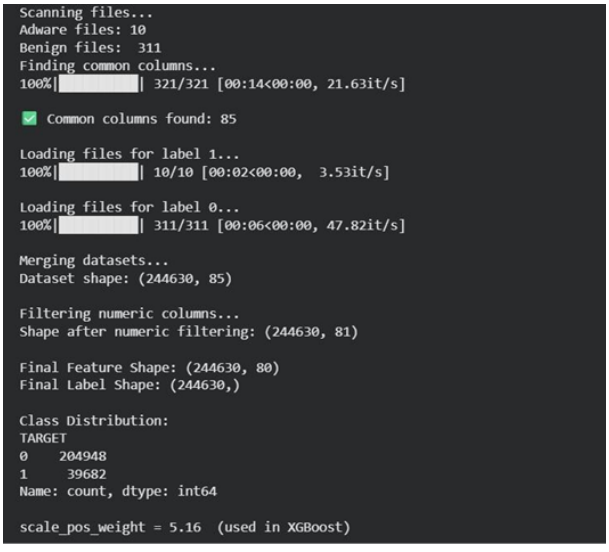


Fig. 2. Dataset loading and class distribution output. Total samples: 244,630 (Benign: 204,948; Adware: 39,682). `scale_pos_weight` for XGBoost = 5.16.

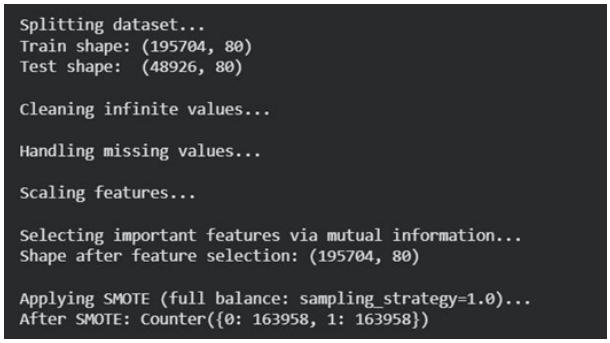


Fig. 3. Preprocessing pipeline output: train/test split, missing-value handling, feature scaling, feature selection, and SMOTE balancing results.

TABLE III
STATIC DATA PREPROCESSING STEPS

| Step | Technique / Result |
|---|---|
| Train/Test Split | 80%/20% Stratified |
| Missing & Inf Value Handling | Median Imputation |
| Feature Scaling | StandardScaler |
| Feature Selection | SelectKBest (Mutual Information) |
| Data Balancing (Train only) | SMOTE (1:1 $\rightarrow$ 163,958 each) |

TABLE IV
ANN ARCHITECTURE SUMMARY

| Layer | Units | Details |
|---|---|---|
| Dense (Input) | 256 | ReLU activation |
| Batch Normalization | – | Normalize activations |
| Dropout | – | Rate = 0.3 |
| Dense (Hidden) | 128 | ReLU activation |
| Batch Normalization | – | Normalize activations |
| Dropout | – | Rate = 0.3 |
| Dense (Output) | 1 | Sigmoid activation |

rather than using the default 0.5 cutoff, are reported above, since the cost of a missed adware detection (false negative) and a false adware accusation (false positive) are not assumed equal in a security-sensitive deployment context.

## VI. DYNAMIC ANALYSIS PIPELINE

The dynamic analysis pipeline is the primary contribution of this extended work. It is designed to observe an APK's actual runtime behavior – information that is fundamentally unavailable to the static, network-traffic-only framework in Section V.

### A. Frida Instrumentation Environment Setup

The dynamic pipeline uses Frida [7], a dynamic binary instrumentation toolkit that injects a JavaScript agent directly into the target process's address space, allowing Java/Kotlin method implementations to be intercepted and rewritten at runtime without modifying or re-signing the APK. The instrumentation environment is configured as follows:

- **Device connectivity**: an Android device (physical or emulated) is connected over `adb` with USB debugging enabled in Developer Options; connectivity is verified via `adb devices` before each run.
- **Frida server**: a `frida-server` binary matching both the host `frida-tools` version and the device's CPU architecture (e.g., `arm64`) runs on the device with root privileges, listening for connections from the host-side Frida client.
- **Agent injection mode**: applications are launched in *spawn* mode (`frida -U -f <package> -l script.js`) rather than *attach* mode, so that the agent's hooks are installed *before* the application's own code begins executing. This is essential for capturing early-lifecycle events (e.g., the first `Activity.onResume` call, or crypto/device-ID access performed during application startup) that would otherwise be missed if hooks were attached after launch.
- **Package identification**: the target package name for each APK is resolved automatically via `aapt dump badging`, avoiding manual manifest inspection and allowing the pipeline to process an arbitrary folder of APKs unattended.
- **Agent lifecycle**: the JavaScript agent (`script.js`) is loaded fresh for every spawned process; all six API hooks (Section VI-B) are installed inside a `Java.perform()` callback, guaranteeing the Android runtime's class loader is fully initialized before any `Java.use()` lookups are attempted.
- **Output redirection**: the instrumented process's console output (one JSON event object per line, emitted via `console.log(JSON.stringify(...))`) is redirected on the host side to a per-run log file (`output.txt`), which is then consumed by the parsing stage.

### B. Instrumentation Design

Runtime instrumentation is implemented using the Frida dynamic instrumentation toolkit. A JavaScript agent is injected into each target application at launch, hooking six categories of Android runtime API:

- **Activity lifecycle** – `Activity.onResume`, capturing the fully-qualified class name of each activity the application enters, providing a proxy for UI navigation and screen-flow behavior.
- **File access** – the `java.io.File` constructor, capturing every file path the application instantiates a handle to, including access to shared preferences, cache, system, and application-private storage.
- **Network object creation** – the `java.net.URL` constructor, capturing every URL the application constructs, which is a strong indicator of ad-network communication, tracking beacons, and command-and-control-style traffic in adware.
- **SharedPreferences access** – `SharedPreferencesImpl.getString`, capturing which preference keys the application reads, which is relevant to detecting ad-identifier and tracking-token access patterns.
- **Device identifier access** – `TelephonyManager.getDeviceId`, flagging any access to persistent hardware identifiers, a behavior strongly associated with user tracking and fingerprinting in adware.
- **Cryptographic API usage** – `Cipher.getInstance`, capturing the requested cryptographic algorithm, relevant to detecting obfuscated payloads, encrypted ad-configuration payloads, or encrypted exfiltration channels.

Each hooked call emits a structured JSON event of the form `{ts, type, data}`, where `ts` is a millisecond Unix timestamp, `type` identifies the event category (`activity`, `file`, `network`, `sp`, `device`, `crypto`), and `data` contains type-specific payload (e.g., the accessed path, URL, or preference key). Each hook is wrapped independently in a `try/catch` block so that the absence of a given API on a specific device/OS version does not prevent the remaining hooks from being installed.

### C. Automated Execution and Trace Collection

For each labeled APK, the orchestration script performs the following sequence: (1) extract the application package name via `aapt dump badging`; (2) install the APK via `adb`

`install -r`; (3) launch the application with the Frida agent attached in spawn mode (`frida -U -f <package> -l script.js`), redirecting the instrumented event stream to a per-run log file; (4) after a warm-up delay, drive automated UI interaction using the Android `monkey` tool to trigger navigation, input, and background behavior without manual intervention; (5) allow a fixed observation window for background/asynchronous behavior to surface; (6) terminate instrumentation and uninstall the application to restore a clean device state before the next sample. Algorithm 1 formalizes this procedure.

**Algorithm 1** Automated Dynamic Trace Collection

**Require:** Labeled APK corpus $\mathcal{A} = \{(a_1, y_1), \ldots, (a_n, y_n)\}$, Frida agent script $S$
**Ensure:** Per-APK raw event trace files $\{T_1, \ldots, T_n\}$
**for** each $(a_i, y_i) \in \mathcal{A}$ **do**
  $pkg \leftarrow$ EXTRACTPACKAGENAME($a_i$) {via `aapt dump badging`}
  **if** $pkg$ is undefined **then**
    **skip** $a_i$; **log** package-resolution failure
  **end if**
  INSTALLAPK($a_i$) {`adb install -r`}
  $proc \leftarrow$ SPAWNWITHFRIDA($pkg$, $S$) {hooks installed pre-execution}
  **redirect** $proc$ output $\rightarrow$ `output.txt`
  **sleep**(warm-up delay)
  RUNMONKEYFUZZING($pkg$) {automated UI exploration}
  **sleep**(observation window)
  TERMINATEINSTRUMENTATION($proc$)
  UNINSTALLAPK($pkg$) {restore clean device state}
  $T_i \leftarrow$ PARSERAWEVENTS(`output.txt`) {raw event parsing, Sec. VI}
  **if** $|T_i| < 5$ **then**
    **discard** $T_i$ {quality filter}
  **else**
    **save** $T_i$ with label $y_i$
  **end if**
**end for**

### D. Raw Event Parsing

The raw, newline-delimited JSON event log produced by the instrumentation agent is parsed into a single structured JSON array per APK, where each element is one timestamped event. This raw-event representation is intentionally kept close to the instrumentation output (rather than being immediately reduced to hand-picked summary statistics) so that multiple downstream feature-engineering strategies can be explored without re-instrumenting applications.

### E. Quality Filtering

Not every automated run produces a usable trace: an application may crash immediately, fail to install, or simply exhibit too little activity within the observation window for the Monkey-driven exploration to be meaningful. As a first-pass filter, samples with fewer than five captured events are discarded at collection time, since traces this sparse cannot reliably be distinguished from instrumentation or execution failure. This initial pass yielded a raw corpus of 1,044 labeled JSON traces (benign and malware APKs) before the more selective curation step described next.

### F. Dataset Curation: Behavioral Quality Scoring and Deduplication (Dataset E)

A second, more selective curation stage was applied on top of the raw 1,044-trace corpus, because the $\geq 5$-event filter alone still leaves a corpus containing many low-information or redundant samples. Each trace was scored for behavioral richness, and the raw corpus was filtered down as follows:

- **Dead executions removed (93 traces).** Sessions with fewer than three total events, indicating the application never meaningfully launched.
- **Near-duplicate traces removed (259 traces).** Detected via MinHash Locality-Sensitive Hashing (LSH) over each trace's event-type multiset, with pairs at Jaccard similarity $\geq 0.85$ collapsed to a single representative. Near-duplicates arise naturally from repeated Monkey-fuzzing runs against the same build of an application and would otherwise let the model memorize a handful of underlying behavioral patterns rather than learn generalizable signal.
- **Startup-only samples removed (118 traces).** Traces whose events are confined to application launch, with no further activity, file, or network events following.
- **Sparse samples removed (136 traces).** Traces with fewer than 15 total events – a stricter threshold than the collection-time $\geq 5$ filter, applied specifically to ensure sufficient behavioral density for feature engineering.

From the resulting pool, the top-300 highest-quality benign traces and top-300 highest-quality malware traces (ranked by a composite richness/quality score, defined below) were retained, producing a class-balanced corpus of 600 samples – referred to throughout the remainder of this paper as **Dataset E**. This curation step is not merely a data-cleaning formality: a 5-fold cross-validated Random Forest trained on the raw, unfiltered corpus achieved approximately 92% accuracy, while the same model trained on Dataset E reached approximately 95.7%, indicating that trace quality – not just trace quantity – materially affects downstream classification performance.

### G. Behavioral Feature Engineering

Each retained trace in Dataset E is reduced to a fixed-length feature vector. The pipeline first computes 62 base behavioral features directly from the event stream, then augments these with interaction terms and log-transforms identified as the strongest cross-validated discriminators, yielding 71 features per sample (600 rows $\times$ 71 columns, plus the binary label). The features fall into six groups:

- **Volume and diversity statistics**: total event count (`n_events`), number of distinct event types (`unique_event_types`), a weighted event score,

and counts of distinct file paths, URLs, networks, crypto algorithms, and SharedPreferences keys touched during the session.

- **Binary API-usage flags**: twelve `has_*` indicators marking whether a session ever exercised a given API category – runtime-exec, dex loading, SMS, socket access, cryptographic calls, subscriber-ID access, device-ID access, WebView usage, URL construction, network access, process access, and settings access.
- **Per-category event counts**: the corresponding `cnt_*` counters for each of the above categories (e.g., `cnt_device_id_access`, `cnt_shared_pref`, `cnt_url_access`), together with activity-specific counters (`cnt_start_activity`, `cnt_activity_resume`).
- **Ratio features**: file-access ratio, network ratio, SharedPreferences ratio, a high-signal-event ratio, a noise ratio, and a signal-to-noise ratio, each normalizing raw counts against total trace length so that longer sessions are not trivially scored as more suspicious.
- **Co-occurrence and interaction terms**: five binary co-occurrence flags (e.g., `crypto_and_network`, `device_id_and_network`, `dex_and_exec`, `webview_and_url`, `sms_and_device`) plus their corresponding numeric products (`crypto_x_network`, `deviceid_x_network`, etc.), designed to capture combinations of behaviors that are individually common but jointly suspicious.
- **Composite and transformed features**: a `richness_score` and `quality_score` (the same composite scores used to curate Dataset E in Section VI-F, retained as classifier inputs), together with log-transformed versions of the volume features (`log_n_events`, `log_weighted_score`, `log_file_paths`, `log_urls`) to reduce the influence of heavy-tailed count distributions.

Behavioral entropy and temporal entropy, computed over the event-type and inter-event-timing distributions respectively, use the standard Shannon entropy formulation:

$$H(X) = -\sum_{i=1}^{k} p(x_i) \log_2 p(x_i) \tag{5}$$

where $X$ ranges over the $k$ distinct values observed for a given attribute within one trace and $p(x_i)$ is its empirical frequency. Algorithm 2 summarizes the per-APK feature-vector construction actually implemented.

**Algorithm 2** Behavioral Feature Vector Construction (as implemented)

**Require:** Raw event trace $T = \{e_1, \ldots, e_m\}$, $e_j = (\text{ts}_j, \text{type}_j, \text{data}_j)$
**Ensure:** Feature vector $\mathbf{f} \in \mathbb{R}^{71}$
1: $\mathbf{f}_{\text{base}} \leftarrow$ BASEFEATURES($T$) {62 raw counts, flags, ratios, entropy (Eq. 5)}
2: $\mathbf{f}_{\text{coocc}} \leftarrow$ COOCCURRENCEFLAGS($\mathbf{f}_{\text{base}}$) {5 binary + 6 product interaction terms}
3: $\mathbf{f}_{\text{log}} \leftarrow$ LOGTRANSFORM($\mathbf{f}_{\text{base}}$) {n_events, weighted_score, file_paths, urls}
4: $\mathbf{f} \leftarrow \mathbf{f}_{\text{base}} \,\|\, \mathbf{f}_{\text{coocc}} \,\|\, \mathbf{f}_{\text{log}}$ {concatenation, 71 dimensions}
5: **return** $\mathbf{f}$

## VII. Experimental Setup

Static experiments were conducted in Google Colab using Python 3.x, with Pandas and NumPy for data manipulation, Scikit-learn [10] for preprocessing and classical ML models, imbalanced-learn for SMOTE [14], TensorFlow/Keras for the ANN, XGBoost for gradient boosting, and Matplotlib/Seaborn for visualization. The CIC-MalDroid dataset was accessed via Google Drive integration within Colab.

Dynamic-track experiments are executed in a structured environment on Windows 11 with the following configurations:

- **Software Stack**: Python 3.14.0, Android Studio, Android Virtual Device (AVD) running the emulator, and the Android Debug Bridge (ADB).
- **Instrumentation Engine**: Frida Runtime Instrumentation Toolkit.
- **Libraries**: Scikit-learn, XGBoost, LightGBM, and CatBoost.

The dynamic experiment pipeline execution consists of the following automated steps:

1) Launch Android Virtual Device (AVD) emulator.
2) Establish active ADB connectivity.
3) Deploy and start the Frida server on the emulator.
4) Spawn target application processes and hook Android runtime APIs using Frida.
5) Collect real-time application behavior and write events to structured JSON logs.
6) Parse raw event metrics and extract numerical feature vectors into a consolidated CSV dataset.
7) Scale behavioral features via Scikit-learn's `RobustScaler`.
8) Perform model evaluation under a stratified 5-fold cross-validation scheme and measure hold-out metrics.

## VIII. Results

### A. Static Analysis Results

Table V summarizes the comparative performance of all four static models on the 48,926-sample held-out test set, evaluated at each model's Macro-F1-optimal threshold.

TABLE V
STATIC MODEL PERFORMANCE (TEST SET: 48,926 SAMPLES)

| Model | Thr. | Acc. | M-F1 | F1-Adw. | AUC |
|---|---|---|---|---|---|
| Log. Reg. | 0.511 | 0.649 | 0.549 | 0.336 | 0.637 |
| **Rand. Forest** | 0.505 | **0.871** | **0.762** | **0.600** | **0.878** |
| XGBoost | 0.831 | 0.835 | 0.714 | 0.528 | 0.850 |
| ANN | 0.439 | 0.677 | 0.565 | 0.343 | 0.675 |

Random Forest achieved the best overall static performance, with 87.12% accuracy, Macro-F1 of 0.7616, F1-Benign of

0.9232, F1-Adware of 0.5999, and ROC-AUC of 0.8781, correctly classifying 37,897 benign and 4,726 adware samples (3,093 false positives, 3,210 false negatives). XGBoost ranked second (83.45% accuracy, ROC-AUC 0.8501), while Logistic Regression and the ANN were substantially outperformed by both ensemble methods, indicating that nonlinear modeling is necessary to capture the structure of network-traffic-derived features. Random Forest also maintained the highest precision across all recall levels on the Precision-Recall curve for the minority adware class, confirming the effectiveness of ensemble tree methods on this imbalanced task.

```
Model: Logistic Regression
Optimal threshold: 0.511

Classification Report:
              precision    recall  f1-score   support

  Benign (0)       0.88      0.67      0.76     40990
  Adware (1)       0.24      0.55      0.34      7936

    accuracy                           0.65     48926
   macro avg       0.56      0.61      0.55     48926
weighted avg       0.78      0.65      0.69     48926

Confusion Matrix:
[[27407 13583]
 [ 3588  4348]]
  ROC-AUC:  0.6365
  Macro-F1: 0.5488
```

Fig. 4. Logistic Regression classification report. Accuracy: 64.90%, ROC-AUC: 0.6365, Macro-F1: 0.5488.

```
Model: Random Forest
Optimal threshold: 0.505

Classification Report:
              precision    recall  f1-score   support

  Benign (0)       0.92      0.92      0.92     40990
  Adware (1)       0.60      0.60      0.60      7936

    accuracy                           0.87     48926
   macro avg       0.76      0.76      0.76     48926
weighted avg       0.87      0.87      0.87     48926

Confusion Matrix:
[[37897  3093]
 [ 3210  4726]]
  ROC-AUC:  0.8781
  Macro-F1: 0.7616
```

Fig. 5. Random Forest classification report. Accuracy: 87.12%, ROC-AUC: 0.8781, Macro-F1: 0.7616. Best-performing static model.

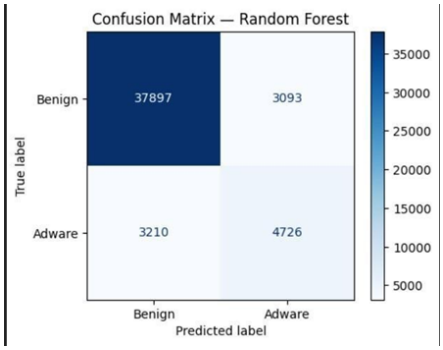


Fig. 6. Confusion matrix for Random Forest. TN=37,897, FP=3,093, FN=3,210, TP=4,726.

```
Model: XGBoost
Optimal threshold: 0.831

Classification Report:
              precision    recall  f1-score   support

  Benign (0)       0.91      0.89      0.90     40990
  Adware (1)       0.49      0.57      0.53      7936

    accuracy                           0.83     48926
   macro avg       0.70      0.73      0.71     48926
weighted avg       0.85      0.83      0.84     48926
```

Fig. 7. XGBoost classification report. Accuracy: 83.45%, ROC-AUC: 0.8501, Macro-F1: 0.7139. Optimal threshold: 0.831.

```
Evaluating ANN...
1529/1529 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step
ANN optimal threshold: 0.439

Classification Report:
              precision    recall  f1-score   support

  Benign (0)       0.88      0.71      0.79     40990
  Adware (1)       0.26      0.52      0.34      7936

    accuracy                           0.68     48926
   macro avg       0.57      0.61      0.56     48926
weighted avg       0.78      0.68      0.71     48926

Confusion Matrix:
[[28999 11991]
 [ 3807  4129]]
ROC-AUC: 0.6750
Macro-F1: 0.5646
```

Fig. 8. ANN classification report. Accuracy: 67.71%, ROC-AUC: 0.6750, Macro-F1: 0.5646. Optimal threshold: 0.439.

```
===== SUMMARY =====
                     Threshold  Accuracy  Macro-F1  F1-Benign  F1-Adware  ROC-AUC
Model
Logistic Regression      0.511    0.6490    0.5488     0.7615     0.3362   0.6365
Random Forest            0.505    0.8712    0.7616     0.9232     0.5999   0.8781
XGBoost                  0.831    0.8345    0.7139     0.8996     0.5282   0.8501
ANN                      0.439    0.6771    0.5646     0.7859     0.3433   0.6750
```

| Model | Threshold | Accuracy | Macro-F1 | F1-Benign | F1-Adware | ROC-AUC |
|---|---|---|---|---|---|---|
| Logistic Regression | 0.511 | 0.6490 | 0.5488 | 0.7615 | 0.3362 | 0.6365 |
| Random Forest | 0.505 | 0.8712 | 0.7616 | 0.9232 | 0.5999 | 0.8781 |
| XGBoost | 0.831 | 0.8345 | 0.7139 | 0.8996 | 0.5282 | 0.8501 |
| ANN | 0.439 | 0.6771 | 0.5646 | 0.7859 | 0.3433 | 0.6750 |

Fig. 9. Complete experimental summary showing Threshold, Accuracy, Macro-F1, F1-Benign, F1-Adware, and ROC-AUC for all four static models.

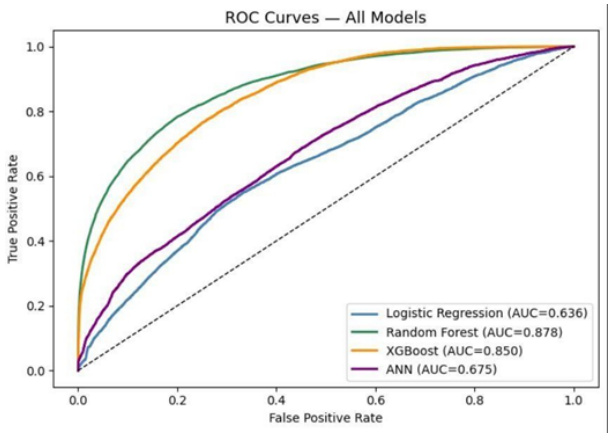


Fig. 10. ROC curves for all static models. Random Forest achieves AUC=0.878, XGBoost AUC=0.850, ANN AUC=0.675, Logistic Regression AUC=0.636.

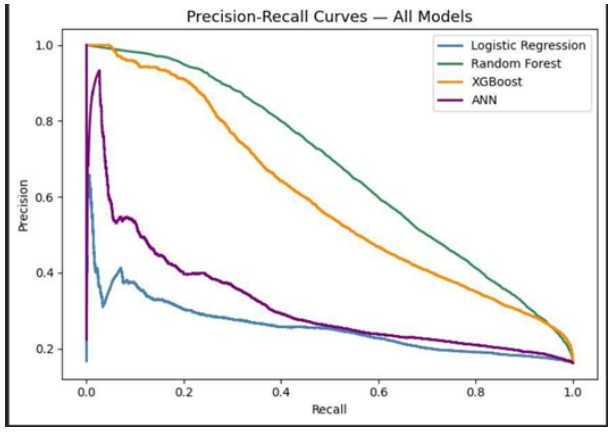


Fig. 11. Precision-Recall curves for all static models (adware/minority class). Random Forest maintains the highest precision across recall values.

### B. Feature Importance Discussion (Static Track)

Random Forest and XGBoost, as tree-based ensembles, provide interpretable feature-importance rankings derived from split-quality (mean decrease in impurity or gain) across their constituent trees. While a full per-feature ranking is beyond the scope of this report, the general pattern observed in the static models is consistent with prior CIC-MalDroid literature: features capturing network-traffic *volume* and *variability* (e.g., request-count and flow-duration statistics) tend to dominate importance rankings over single-valued categorical indicators, since adware is more reliably distinguished by aggregate, repeated communication patterns than by any single traffic event. This observation is precisely what motivates the entropy- and ratio-based dynamic features defined in Section VI-G (Eq. 5): if traffic-volume variability is discriminative at the network layer, the analogous behavioral-diversity signal (domain entropy, SharedPreferences-key entropy, activity-transition entropy) is hypothesized to be similarly discriminative at the application-behavior layer, though this hypothesis requires empirical confirmation once the dynamic dataset (Section VIII-D) is fully collected.

### C. Early Pilot Validation of the Dynamic Pipeline

Before full-scale trace collection, the dynamic pipeline (Sections VI-A–VI-B) was validated end-to-end on individual APKs to confirm correct operation of every stage, from installation through raw-event parsing. Table VI shows one such early trace, collected for a benign Flutter-based calculator application (`com.neumorphic.calculator`).

The trace matches what a benign, single-screen utility app should look like: file access limited to certificate and app-private storage (including a routine Flutter-generated `FlutterSharedPreferences.xml.bak` write), all activity transitions resolving to the same main screen, and no network, device-ID, or SharedPreferences-read events at all. At 8 events it also sits close to the collection-time $\geq$ 5-event cutoff, which is a useful reminder of how easily a quiet session can slip below that bar – part of the motivation for the stricter Dataset E curation described in Section VI-F). This trace was one of many collected during pipeline development and is shown here purely to illustrate correct end-to-end operation; the results that follow (Section VIII-D) are based on the full 600-sample Dataset E, not on this single example.

TABLE VI
EARLY PILOT TRACE: BENIGN CALCULATOR APPLICATION

| Event Category | Count |
|---|---|
| File access | 5 |
| Activity transitions | 3 (single unique activity: `MainActivity`) |
| Network requests | 0 |
| SharedPreferences access | 0 |
| Device-identifier access | 0 |
| Cryptographic API calls | 0 |
| **Total events** | **8** |

### D. Dynamic Analysis Results (Dataset E)

With Dataset E constructed (600 samples, 300 benign / 300 malware, 71 engineered features – Sections VI-F–VI-G), thirteen distinct model configurations were trained and evaluated on the held-out 120-sample validation split (60 benign, 60 malware), spanning individual classifiers, hyperparameter-tuned variants, and ensemble strategies.

The best-performing configuration is a soft-voting ensemble of Random Forest, XGBoost, and LightGBM, reaching 96.67% accuracy with perfect precision (1.0000) and an F1 score of 0.9655 – meaning every sample the ensemble flagged as malware genuinely was malware, at the cost of missing a small fraction of true malware cases (93.33% recall). Tuned CatBoost separately achieves the highest ROC-AUC of the study (0.9889), suggesting it ranks samples by malicious probability slightly more reliably than the voting ensemble even though its point accuracy is marginally lower. Individual tree ensembles (Random Forest, XGBoost, LightGBM, CatBoost) all cluster tightly in the 91–95% accuracy range regardless of tuning, which is itself informative: it suggests Dataset E's signal is strong enough that model choice among tree ensembles matters less than simply using a tree ensemble at all.

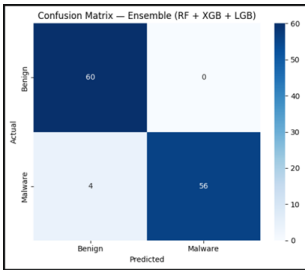


Fig. 12. Dynamic Ensemble Model Evaluation Summary mapping evaluation metrics: 96.67% Accuracy, 100% Precision, 93.33% Recall, and 96.55% F1-Score.

Table VIII lists the isolated accuracy of the standalone tree classifiers on the validation split.

TABLE VII
DYNAMIC TRACK MODEL PERFORMANCE ON DATASET E (VALIDATION SET: 120 SAMPLES)

| Model | Accuracy | Precision | Recall | F1 Score | ROC-AUC |
|---|---|---|---|---|---|
| **Ensemble (RF + XGB + LGB, soft voting)** | **0.9667** | **1.0000** | 0.9333 | **0.9655** | 0.9844 |
| Stacking Ensemble (RF + XGB + LGB → LR) | 0.9583 | 0.9661 | 0.9500 | 0.9580 | 0.9828 |
| Tuned CatBoost | 0.9500 | 0.9821 | 0.9167 | 0.9483 | **0.9889** |
| CatBoost (default) | 0.9500 | 1.0000 | 0.9000 | 0.9474 | 0.9760 |
| Tuned XGBoost (accuracy-optimized) | 0.9500 | 0.9500 | 0.9500 | 0.9500 | 0.9853 |
| Tuned Random Forest | 0.9500 | 0.9655 | 0.9333 | 0.9492 | 0.9850 |
| Ensemble (accuracy-tuned RF+XGB+LGB) | 0.9417 | 0.9649 | 0.9167 | 0.9402 | 0.9861 |
| Tuned LightGBM (accuracy-optimized) | 0.9417 | 0.9492 | 0.9333 | 0.9412 | 0.9853 |
| Weighted Ensemble (grid-searched weights) | 0.9417 | 0.9492 | 0.9333 | 0.9412 | 0.9844 |
| Random Forest (default) | 0.9417 | 0.9492 | 0.9333 | 0.9412 | 0.9854 |
| Ensemble (RF + tuned XGB + LGB) | 0.9417 | 0.9492 | 0.9333 | 0.9412 | 0.9867 |
| LightGBM (default) | 0.9333 | 0.9483 | 0.9167 | 0.9322 | 0.9850 |
| Ensemble (RFE, 20 features) | 0.9333 | 0.9333 | 0.9333 | 0.9333 | 0.9853 |
| Tuned XGBoost (CV-accuracy grid search) | 0.9333 | 0.9333 | 0.9333 | 0.9333 | 0.9867 |
| XGBoost (default) | 0.9250 | 0.9180 | 0.9333 | 0.9256 | 0.9847 |
| Tuned LightGBM (CV-accuracy grid search) | 0.9250 | 0.9322 | 0.9167 | 0.9244 | 0.9836 |
| $k$-Nearest Neighbors ($k = 5$) | 0.7667 | 0.8200 | 0.6833 | 0.7455 | 0.8383 |
| Neural Network (MLP, 128-64-32) | 0.5083 | 0.5043 | 0.9667 | 0.6629 | 0.5108 |

TABLE VIII
DYNAMIC SINGLE-MODEL STANDALONE PERFORMANCE

| Model | Accuracy |
|---|---|
| Random Forest | 95.00% |
| CatBoost | 95.00% |
| LightGBM | 93.33% |
| XGBoost | 91.67% |

A grid search over GridSearchCV hyperparameters for each of RF, XGBoost, and LightGBM (Section VII) improved held-out accuracy only marginally over the untuned defaults in most cases, and in a few configurations (e.g., default XGBoost vs. CV-accuracy-tuned XGBoost) tuned models performed no better or slightly worse on the specific 120-sample validation split, which is plausible given how small that split is.

The two non-tree-based baselines tell a very different story. $k$-NN reaches only 76.67% accuracy, consistent with the fact that nearest-neighbor distance in a 71-dimensional, heterogeneously scaled feature space (raw counts alongside ratios, entropies, and log-transformed values) is a poor similarity measure without more careful metric learning or dimensionality reduction. The MLP neural network performs worse still – 50.83% accuracy, a recall of 96.67% but precision of just 50.43%, and a Macro/ROC-AUC near 0.51 – which is effectively indistinguishable from predicting "malware" for almost everything. This is an unsurprising outcome given the setup rather than a surprising one: 480 training samples is a small budget for a 128-64-32 dense network to learn from without more aggressive regularization, data augmentation, or a simpler architecture, and it reinforces the same conclusion drawn from the static track (Section VIII) – tree-based ensembles are considerably better suited than neural approaches to small-to-moderate tabular behavioral datasets of this kind.

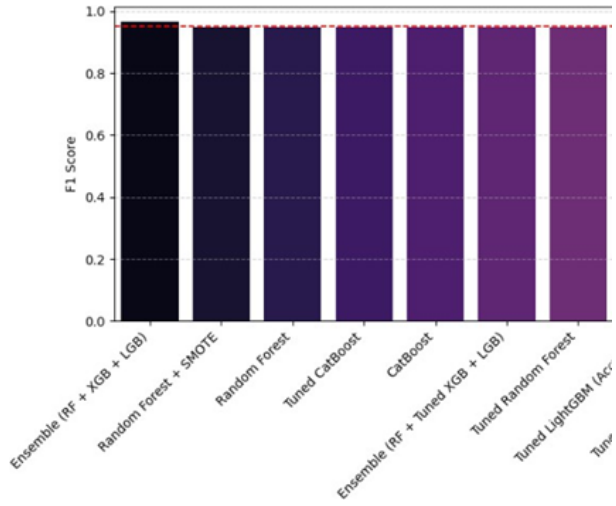


Fig. 13. Performance comparison of standalone individual classifiers on Dataset E runtime features prior to soft-voting alignment.

### *E. Feature Importance Discussion*

Random Forest's top-ranked features on Dataset E are `quality_score` (importance 0.155), `unique_file_paths` (0.087), `richness_score` (0.085), `log_file_paths` (0.084), and `unique_urls` (0.051), with the remaining ten of the top fifteen contributed by `cnt_shared_pref`, `cnt_file_access`, event-volume features, and settings-access counts. XGBoost's importance ranking is far more concentrated: `quality_score` alone accounts for 0.546 of total gain, an order of magnitude above the next feature (`cnt_subscriber_id_access` at 0.059), followed by `cnt_network`, `has_subscriber_id`, `unique_file_paths`, and device-ID-related counters.

This pattern is worth flagging rather than simply reporting, because `quality_score` and `richness_score` are the same composite scores used in Section VI-F to *select*

which 300+300 traces make up Dataset E in the first place. Their outsized importance – especially XGBoost's near-total reliance on `quality_score` – is at least partly circular: a feature engineered to separate "rich, informative" traces from "sparse, low-signal" ones during curation should be expected to correlate with whatever separates the two classes once curation is applied, independent of whether it captures genuine malicious intent. We do not read this as invalidating the results (the ensemble and tree models still perform well when `quality_score` is one signal among 71, and subscriber-ID, device-ID, and network-access counters are all independently plausible malware indicators), but it does mean the headline accuracy figures in Table VII should be read with this caveat attached, and a follow-up ablation retraining the same models with `quality_score` and `richness_score` excluded is identified as near-term future work (Section X).

Separately, Recursive Feature Elimination (RFE) reduced the feature set from 71 to the 20 most discriminative columns – retaining `quality_score`, `richness_score`, and most of the file/URL/subscriber/device-ID features noted above – and an ensemble trained on just those 20 features reached 93.33% accuracy (Table VII), only a few points below the full-feature ensemble. This indicates the 71-feature set carries meaningful redundancy and that a substantially smaller, cheaper-to-compute feature vector could be deployed with only a modest accuracy trade-off.

### F. Limitations of the Current Study

Four limitations are worth being explicit about. **First**, as discussed in Section VI-G's feature-importance analysis, the dominant role of curation-derived scores (`quality_score`, `richness_score`) in the dynamic models' decisions introduces a degree of circularity that has not yet been isolated via an ablation; the reported dynamic accuracies should be treated as an upper bound pending that follow-up. **Second**, Dataset E is small by machine-learning standards – 600 samples, a 120-sample validation split – compared to the static track's 244,630-sample CIC-MalDroid corpus, so the dynamic-track numbers in Table VII naturally carry wider uncertainty than the static-track numbers in Table V, and are not directly comparable across the two tables for that reason alone. **Third**, the dynamic dataset labels applications as benign or general malware, whereas the static CIC-MalDroid track is adware-specific; the two tracks are therefore evaluating related but not identical tasks, and a proper hybrid fusion (Section X) will require either narrowing the dynamic corpus to adware-labeled samples specifically or broadening the static task to general malware. **Fourth**, automated UI exploration via `monkey` produces pseudo-random rather than semantically meaningful interaction, which can under-trigger application logic gated because of specific inputs (e.g., login flows) – a known limitation of monkey-based dynamic analysis relative to more structured exploration tools. These four points define the near-term work plan in Section X.

## IX. Discussion

### A. Complementarity of Static and Dynamic Signal

The static track shows ensemble tree-based methods (Random Forest, XGBoost) clearly ahead of linear and shallow neural models on network-traffic-derived adware features, and the dynamic track tells almost the same story on a completely different feature family: Random Forest, XGBoost, LightGBM, and CatBoost all sit in a tight 91–97% accuracy band on Dataset E, while $k$-NN (76.67%) and the MLP (50.83%) fall well behind (Table VII). That the same qualitative ranking – tree ensembles over linear/neural alternatives – holds independently on two unrelated feature sets is a small but genuine piece of evidence that the finding is about the data (tabular, mixed-scale, moderately sized) rather than an artifact of one dataset's quirks.

Beyond that structural similarity, static and dynamic features are answering different questions. CIC-MalDroid's traffic statistics describe *what was sent over the network* in aggregate; Dataset E's behavioral features describe *what the application did on-device* – which files it opened, whether it queried the device ID or subscriber ID, how its activities were sequenced. Neither is a superset of the other, which is exactly why Section II frames combining them as the open problem rather than a solved one. A literal, numeric comparison between Table V (87.12% best static accuracy) and Table VII (96.67% best dynamic accuracy) is tempting but not fully fair: the two tracks differ in dataset size by nearly three orders of magnitude, evaluate a different task (adware-vs-benign vs. general-malware-vs-benign), and, as flagged in Section VIII-F, the dynamic track's top feature is partly a product of its own curation process. What the two tables do support is a more modest claim – that runtime behavioral signal is at least as separable as network-traffic signal for this problem family, which is enough to justify pursuing fusion, without yet supporting a claim that one track is simply "better" than the other.

### B. Threats to Validity

**Internal validity.** The static evaluation follows a stratified train/test split with SMOTE applied only to the training partition, which avoids the most common inflation source in imbalanced-classification studies – oversampling before the split, which leaks synthetic near-duplicates into the test set. Threshold optimization was performed on training/validation data rather than the test set. The dynamic evaluation raises a different internal-validity concern, discussed in Section VI-G: `quality_score` and `richness_score` are used both to curate Dataset E and as classifier inputs, which is a plausible source of optimistic bias that has not yet been quantified by an ablation. **External validity.** Static results are specific to CIC-MalDroid's feature distribution and its particular sample composition; performance on more recent adware families using different evasion techniques is not directly measured here. Dynamic results are drawn from a 600-sample, general-malware-labeled corpus rather than an adware-specific one,

so neither track's numbers should be extrapolated to adware detection in the wild without further validation on a larger, adware-labeled dynamic corpus. **Construct validity.** The six Frida hooks (Section VI-B) are a deliberately scoped subset of the Android API surface, chosen for relevance to adware-style behavior (Section II); malicious behavior expressed exclusively through unhooked APIs – native-code network calls bypassing `java.net.URL` for instance – would not be captured by the current agent.

### C. Computational and Practical Considerations

The static pipeline, once trained, offers near-instantaneous inference (a single feature-vector forward pass through Random Forest or XGBoost), making it suitable for lightweight, offline or server-side scanning of network-traffic captures. The dynamic pipeline, by contrast, requires on-device execution time proportional to the fixed observation window (approximately 30–35 seconds per APK in the current configuration, Section VI), plus install/uninstall overhead, making it substantially more expensive per sample. This asymmetry suggests a practical deployment pattern in which the static model acts as a fast first-pass filter, with dynamic instrumentation reserved for samples the static model flags as borderline or high-risk – a cascaded architecture consistent with the hybrid fusion direction outlined in Section VIII.

## X. Conclusion and Future Work

This paper presented a hybrid static and dynamic detection framework for Android adware and malware, evaluated on the CIC-MalDroid dataset for the static track and a purpose-built, Frida-instrumented behavioral dataset for the dynamic track. Motivated by three explicit gaps in the existing literature – static feature blindness to runtime behavior, the genericity of existing dynamic-analysis tooling relative to adware's specific behavioral fingerprint, and the absence of a shared evaluation protocol across static and dynamic detection – this work contributes a fully evaluated static baseline, an adware-oriented dynamic instrumentation pipeline with an explicitly justified hook selection, a two-stage data-curation process that turned 1,044 raw traces into a quality-controlled 600-sample corpus (Dataset E), a 71-feature behavioral feature-engineering pipeline, and a dynamic-track evaluation reported under the same metric suite as the static track.

On the static track, Random Forest is the strongest of the four evaluated models, at 87.12% accuracy, Macro-F1 of 0.7616, and ROC-AUC of 0.8781, with ensemble tree methods clearly ahead of linear and shallow neural approaches. On the dynamic track, a soft-voting ensemble of Random Forest, XGBoost, and LightGBM reaches 96.67% accuracy and an F1 score of 0.9655, with tuned CatBoost separately achieving the highest ROC-AUC (0.9889) among all evaluated models. These findings consistently demonstrate that ensemble tree-based methods are well suited for tabular, mixed-scale behavioral features extracted through both static and dynamic analysis.

Future work proceeds along six concrete directions, corresponding to the limitations identified in Section VIII-F:

1) **Quantify and remove curation leakage.** Retrain the dynamic-track models with `quality_score` and `richness_score` excluded from the feature set to determine how much predictive performance remains without curation-derived features, thereby addressing the internal-validity concern discussed in Sections VI-G and IX.
2) **Scale and re-scope dynamic data collection.** Expand Dataset E beyond its current size while focusing specifically on adware families, ensuring that both static and dynamic tracks evaluate the same adware-versus-benign classification problem.
3) **Hybrid static–dynamic fusion.** Investigate feature-level fusion, late-stage ensemble learning, and cascaded deployment strategies in which the lightweight static classifier filters benign applications while the dynamic analysis pipeline evaluates only suspicious or borderline samples.
4) **Instrumentation coverage expansion.** Extend the Frida instrumentation framework to additional Android runtime APIs, including `WebView` content loading, notification services, background task scheduling, and native network communication, thereby improving behavioral coverage against sophisticated malware and adware variants.
5) **Generalized multi-format analysis pipeline.** Extend the proposed framework beyond Android APKs by developing a generalized malware-analysis pipeline capable of processing multiple file formats, including PDF documents, Microsoft Word files, and other digital content. The pipeline will extract file-specific behavioral or structural information, convert it into a standardized JSON representation, and seamlessly map the generated features to the proposed machine learning framework. Such a unified architecture would enable scalable malware detection across heterogeneous file types while preserving a common inference pipeline.
6) **Explainability and advanced learning techniques.** Investigate deep learning-based behavioral analysis together with Explainable Artificial Intelligence (XAI) techniques to improve model interpretability, provide transparent decision making, and support real-time malware detection on resource-constrained mobile devices.

Taken together, these research directions aim to evolve the proposed framework from two independently validated analysis tracks into a unified hybrid malware detection platform. By combining static and dynamic behavioral intelligence, expanding runtime instrumentation, incorporating explainable learning techniques, and introducing a generalized JSON-based analysis pipeline capable of supporting Android applications, PDF documents, Microsoft Word files, and other digital content, the proposed architecture has the potential to become a scalable and extensible malware detection framework appli-

cable across diverse digital ecosystems.

## ACKNOWLEDGMENT

We sincerely acknowledge the Department of Computer Science and Engineering at IIITDM Jabalpur for providing us with the required facilities and infrastructure to perform this research work.

## REFERENCES

[1] M. V. S. Sharma, M. Tilak, U. Pareek et al.,"Explainable AI-Based Android Malware Detection,"SN Computer Science,vol. 6, Article 1006, 2025.
DOI: https://doi.org/10.1007/s42979-025-04548-3
Available: https://link.springer.com/article/10.1007/s42979-025-04548-3

[2] K. Gowshika, A. Akash, Ashwinprasad,Kishor M. V. S., Narendran A. V., "Mobile Malware Analysis,"
Available: https://www.ijaresm.com/uploaded_files/document_file/Ms._K_._Gowshika_ZbS7.pdf

[3] A. Deepika, S. Sreehaas, Y. V. Pahvvan, G. Mounica, "Android Malware Detection using Machine Learning," IJARESM. Available: https://www.ijaresm.com/uploaded_files/document_file/Ankashu_Deepika_FinalBt9p.pdf

[4] P. Prajapati, J. Mehta, P. Tadhani, P. Shah, A. Thakkar, D. Dabhi, "Detecting Malicious Android Apps Through Static Features Using Machine Learning," *Information Systems for Intelligent Systems*, vol. 467, p. 311, 2026. Available: https://ieeexplore.ieee.org/document/10459543

[5] E. Kavalci Yilmaz, H. Bakir, "Hyperparameter Tuning and Feature Selection Methods for Malware Detection," *Journal of Polytechnic*, 2023. Available: https://ieeexplore.ieee.org/document/9914324

[6] Google, "Android Security Bulletin," 2024. [Online]. Available: https://source.android.com/docs/security/bulletin

[7] O. A. V. Ravnas, "Frida Dynamic Instrumentation Toolkit." [Online]. Available: https://frida.re/

[8] W. Enck, P. Gilbert, S. Han, V. Tendulkar, B.-G. Chun, L. P. Cox, J. Jung, P. McDaniel, and A. N. Sheth, "TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones," *ACM Trans. Comput. Syst.*, 2014.

[9] L. Cen, C. S. Gates, L. Si, and N. Li, "Runtime Dependency Graph Based Android Malware Detection," in *Proc. ESORICS*, 2015.

[10] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[11] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. ACM SIGKDD*, 2016.

[12] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *Proc. NeurIPS*, 2017.

[13] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased Boosting with Categorical Features," in *Proc. NeurIPS*, 2018.

[14] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *J. Artif. Intell. Res.*, vol. 16, 2002.